

Ciclo de vida del software

Etapas del ciclo de vida de un software

1. **Análisis:** Definir y documentar requisitos funcionales y no funcionales del software. En esta etapa se dan las entrevistas con el cliente.
2. **Diseño:** Elaborar la arquitectura del software y crear un diseño detallado, definiendo componentes del sistema, UI, etc, que será la guía en el desarrollo.
3. **Desarrollo:** Codificación de los algoritmos y estructuras de datos definidas anteriormente,
4. **Pruebas:** Pruebas exhaustivas para verificar que el software cumple requisitos y funciona correctamente. Tipos: de unidad, integración, sistema, aceptación de usuario, etc.
5. **Implementación:** Puesta en funcionamiento del sistema.
6. **Mantenimiento y evolución:** Se agregan nuevas funcionalidades (evolución) y se corrige errores que puedan surgir (mantenimiento), mejorando y adaptando el software a nuevos requerimientos o tecnologías.

Modelos de ciclo de vida

Los distintos modelos de ciclo de vida se diferencian por tres puntos:

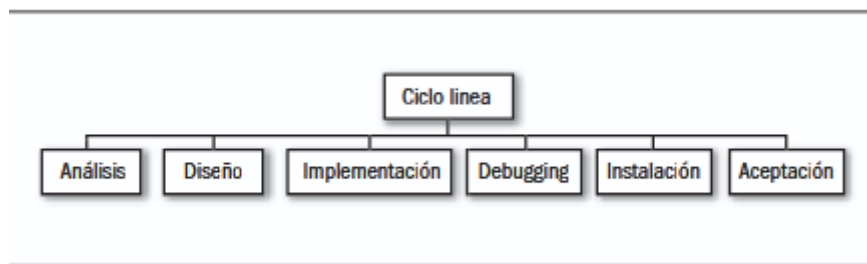
- **El alcance:** saber si es viable el desarrollo de un producto, el desarrollo completo o el desarrollo completo más las actualizaciones y el mantenimiento.
- **La calidad y cantidad de las etapas** en que se divide el ciclo según el tipo de ciclo que se use y el proyecto.
- **La estructura y la sucesión de las etapas**, si hay retroalimentación entre ellas y si es posible repetirlas.

El *riesgo* se refiere a la probabilidad que existe de tener que retomar una etapa anterior y, así, perder tiempo, dinero y esfuerzo. Este es inherente a cualquier modelo y se intenta estar preparados para los posibles cambios o problemas.

Ciclo de vida lineal

Se descompone la actividad global del proyecto en etapas separadas que se realizan de forma lineal, volviendo sencilla la división de tareas y previsión de los tiempos.

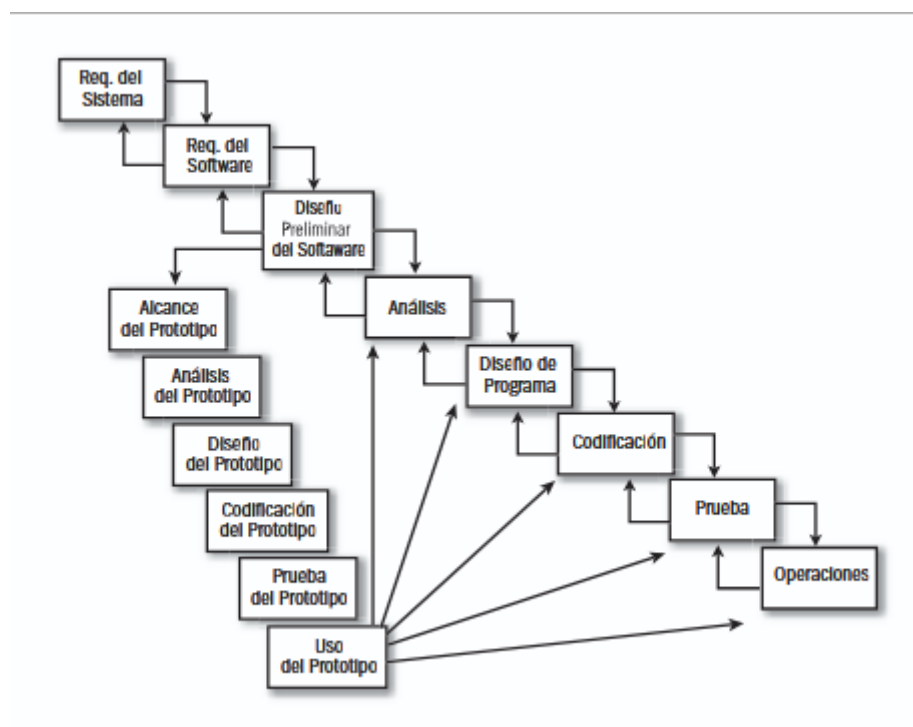
Es necesario que las etapas sean independientes entre sí y debe conocerse lo que va a ocurrir en cada una de ellas antes de comenzarlas. Así, se minimizan las posibilidades de errores en la codificación y reduce la necesidad de requerir información del cliente.



Es sencillo en su gestión y administración económica y temporal. Se recomienda en pequeños proyectos ABM, pero no es apto para desarrollos que requieran retroalimentación entre etapas ya que se vuelve costoso retomar una etapa anterior.

Ciclo de vida en cascada puro

Luego de cada una de las etapas se realizan revisiones para asegurarse de que se puede pasar a la siguiente. Es un modelo rígido y restringido.



Ventajas: Es de planificación sencilla y provee un elevado grado de calidad.

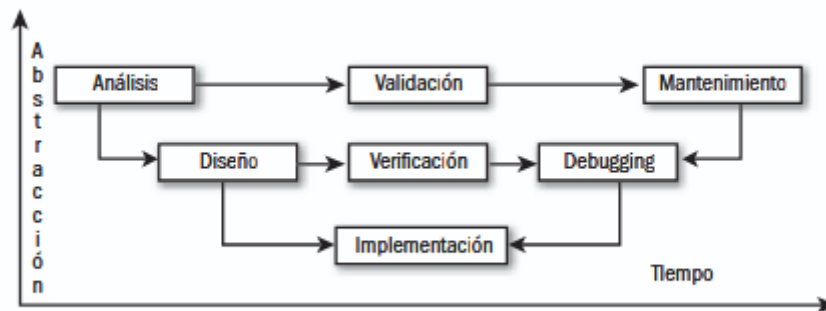
Desventajas: se necesitan todos los requerimientos al comienzo del proyecto, si se cometen errores que no se descubren en la etapa inmediata siguiente, es costoso y difícil volver para realizar la corrección. Además, los resultados no se ven hasta la etapa final.

Es un ciclo adecuado para proyectos que disponen de los requerimientos o funcionalidades desde el inicio o, bien, se entienden perfectamente.

Tipos: medianos y grandes proyectos.

Ciclo de vida en V

Contiene las mismas etapas que el ciclo de vida en cascada puro, pero agregando dos subetapas de retroalimentación: la **validación** entre *análisis* y *mantenimiento*, y la **verificación** entre *diseño* y *debugging*.



Las ventajas y desventajas son las mismas que en el ciclo anterior, añadiendo que los controles cruzados entre las etapas logran una mayor corrección.

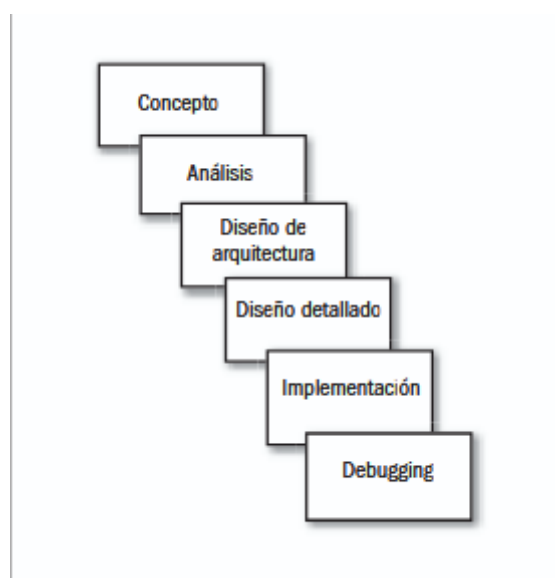
Su uso se recomienda en aplicaciones pequeñas que necesitan alta confiabilidad.

Tipo: medianos y grandes.

Ciclo de vida tipo Sashimi

Es similar al ciclo de vida en cascada puro, pero en este se pueden solapar las etapas. Esto aumenta la eficiencia, ya que la retroalimentación es implícita del modelo.

El solapamiento de las etapas permite utilizar el **modelo de tres capas**: busca separar la arquitectura en tres capas: de **datos**, donde solo se almacenan; de **negocios**, donde se colocan todas las transacciones y validaciones; y de **presentación**, donde se encuentran las vistas y la interacción con el usuario.



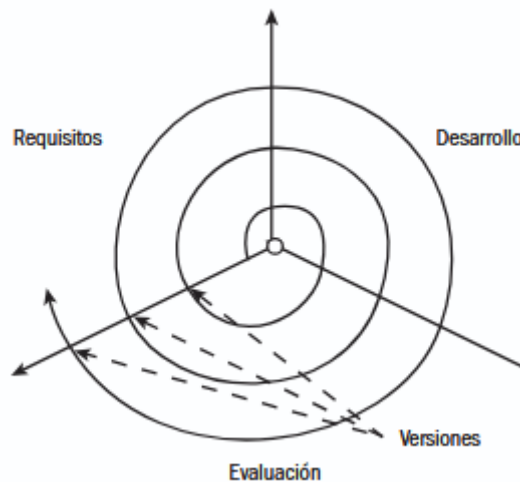
Se gana calidad en el producto final y no es necesaria una documentación detallada, sin embargo, el solapamiento dificulta gestionar el inicio y fin de cada etapa y, los problemas de comunicación, si existen, generan inconsistencias en el proyecto.

Se recomienda en aplicaciones que compartirán recursos con otras aplicaciones en producción.

Tipo: medianos y grandes.

Ciclo de vida evolutivo

Utiliza una iteración de ciclos **requerimientos-desarrollo-evaluación** para responder al hecho de que los requerimientos del usuario pueden cambiar en cualquier momento.



Ventajas: luego del desarrollo se obtiene una nueva versión del producto, no hace falta conocer la mayoría de los requerimientos.

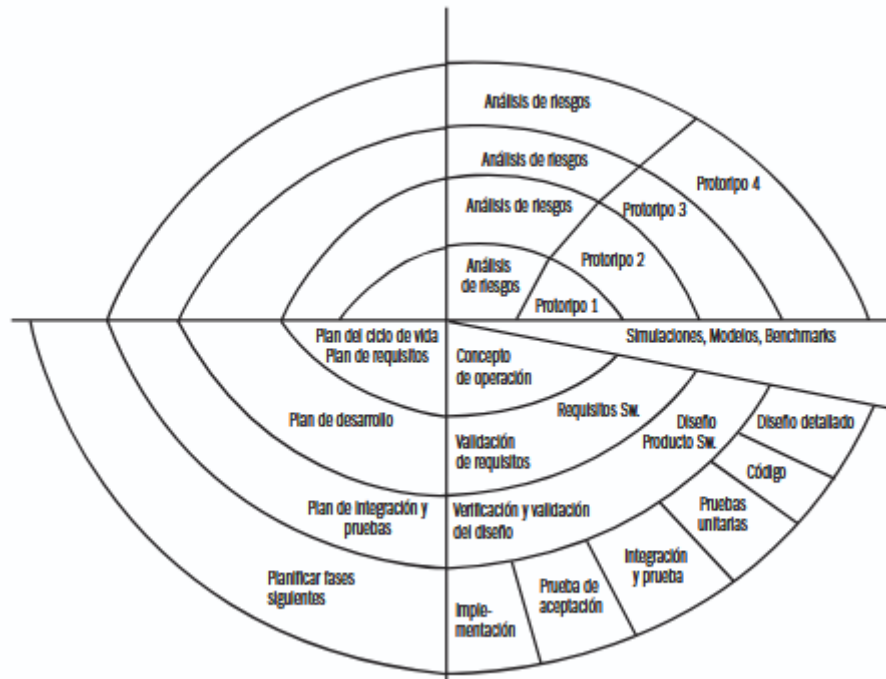
Tipo: pequeños y medianos.

Ciclo de vida en espiral

Se basa en una serie de ciclos repetitivos para ganar madurez en el proyecto final; a medida que el ciclo se cumple, se obtienen prototipos sucesivos para presentar al cliente.

En este modelo hay cuatro actividades que envuelven las etapas:

- **Planificación:** Relevamiento de requerimientos iniciales o luego de una iteración.
- **Análisis de riesgo:** Según el relevamiento se decide si se continúa o no con el desarrollo.
- **Implementación:** Se genera un prototipo basado en los requerimientos.
- **Evaluación:** El cliente evalúa el prototipo y, si presta conformidad, se termina el proyecto. Caso contrario, se incluyen los nuevos requerimientos solicitados por el cliente en la próxima iteración.



Ventajas: permite que el proyecto comience sin tener en claro los requerimientos y tiene bajo riesgo de demoras en caso de detección de errores, ya que pueden resolverse en la próxima rama del espiral.

Desventajas: tiene un alto coste temporal y es difícil evaluar los riesgos y la necesidad de comunicación continua con el cliente.

Tipo: grandes.

Comparación entre ciclos

Ciclo	Características	Proyectos
Lineal	Sencillo en gestión y administración económica-temporal.	Pequeños.
Cascada puro	Sencillo en planificación, poco flexible, validación tardía.	Medianos y grandes.
Espiral	Se comienza con alto riesgo de incertidumbre y cuenta con costos elevados.	Grandes.
Evolutivo	Acepta cambios de requerimientos por parte del usuario.	Pequeños y medianos.
Sashimi	Solapamiento de etapas, sirve cuando desde el comienzo no se tiene toda la información.	Medianos y grandes.
En V	Sencillo en planificación, poco flexible, validación tardía. Se agregan dos subetapas: validación (entre análisis y mantenimiento) y verificación (entre diseño y debugging).	Medianos y grandes.

Técnicas de recolección de información

La recolección de información se realiza en la primera etapa de desarrollo, el **análisis de requisitos**, identificando necesidades, expectativas y restricciones del cliente. Es vital para asegurar que el software cumpla con las expectativas, reduciendo errores y costos por correcciones.

Métodos de recolección:

1. **Entrevistas:** Preguntas al cliente para obtener detalles de sus necesidades.
2. **Cuestionarios:** Preguntas diseñadas para recopilar datos rápidos de un grupo amplio de usuarios (abiertas o cerradas, dispuestas piramidalmente).
3. **Observación directa:** Analizar cómo los usuarios realizan tareas en su entorno real para identificar problemas y oportunidades.
4. **Análisis de documentación:** Revisar documentación existente (manuales, informes) para comprender el contexto sin depender de entrevistas.
5. **Tormenta de ideas:** Grupo propone ideas y requisitos sin restricciones para fomentar la creatividad y explorar enfoques.
6. **Prototipos:** Modelos preliminares del software para obtener retroalimentación del usuario.

Recopilación de información: métodos interactivos

Entrevistas

La entrevista es una conversación dirigida con preguntas y respuestas para obtener información sobre HCI, estética, usabilidad, utilidad, objetivos, sentimientos, opiniones y procedimientos de toma de decisiones.

Los **cinco pasos para planearla** son: leer antecedentes, establecer objetivos, decidir a quién entrevistar, preparar al entrevistado y definir tipo y estructura de preguntas.

Las **preguntas** pueden ser: **abiertas** (respuesta libre), **cerradas** (opciones limitadas) o de **sondeo** (seguimiento para más detalles, pueden ser abiertas o cerradas).

La **estructura** puede ser: **pirámide** (de preguntas específicas a generalizadas), **embudo** (de abiertas a específicas) o **diamante** (combina las dos anteriores, requiere más tiempo).

Cuestionarios

Los cuestionarios recopilan información sobre HCI, posturas, creencias, comportamiento y características de personas clave, siendo útiles en organizaciones dispersas, con mucha gente o si se requiere trabajo exploratorio. Las **preguntas** deben ser simples, específicas, cortas, objetivas, precisas, dirigidas al personal con conocimiento y en un nivel de lectura adecuado. Escalar los cuestionarios puede medir posturas o características de los encuestados.

Recopilación de información: métodos discretos

Muestreo

El muestreo es la selección sistemática de elementos representativos de una población (documentos, miembros de la organización). Su objetivo es reducir costos, agilizar la recolección de información y mejorar la efectividad del estudio.

Diseñar una muestra implica: determinar la población, elegir el tipo de muestra, calcular su tamaño y planificar la recolección de datos.

Los tipos de muestras son: de **conveniencia** (sin restricción), **intencionadas** (por juicio), **simples aleatorias** (por lista numerada) y **complejas aleatorias** (sistemático, estratificado o por conglomerado).

Investigación

Se deben investigar los datos y formularios actuales y archivados, que revelan de dónde viene la organización y hacia dónde va. Se debe analizar tanto los documentos cuantitativos como los cualitativos, ya que estos son mensajes persuasivos de la propia organización.

Observación de comportamiento en la toma de decisiones

Por medio de ella se llega a entender lo que en realidad ocurre cuando el usuario interactúa con las tecnologías de la información.

Observación del entorno físico

Se buscan pistas que indiquen qué tan bien se adapta el sistema al usuario, entre otras cuestiones de HCI.

Ingeniería de requerimientos

Un **requerimiento** es una descripción de lo que el sistema debe hacer: el servicio que ofrece y las restricciones en su operación. Estos reflejan las necesidades de los clientes sobre el sistema. Al proceso de descubrir, analizar, documentar y verificar estos servicios y restricciones se le llama **ingeniería de requerimientos (IR)**.

Una forma de clasificarlos es según su nivel de **abstracción**, donde se obtienen:

1. **Requerimientos del usuario:** qué servicios esperan los usuarios del sistema, y de las restricciones con las cuales éste debe operar.
2. **Requerimientos del sistema:** funciones, servicios y restricciones operacionales del sistema de software.

Otra forma de clasificación es según su **naturaleza**, donde se hallan:

1. **Requerimientos funcionales:** servicios que el sistema debe proveer, de cómo debería reaccionar a entradas particulares y de cómo debería comportarse en situaciones específicas.
2. **Requerimientos no funcionales:** limitaciones sobre servicios o funciones que ofrece el sistema, cómo la temporización y el proceso de desarrollo o imposiciones de los estándares.

Requerimientos funcionales

Refieren a lo que el sistema debe hacer y dependen del tipo de software que se esté desarrollando, de los usuarios esperados y del enfoque general que adopta la organización al escribir los requerimientos.

Al expresarse como **requerimientos de usuario** detallan las funciones del sistema, sus entradas y salidas, excepciones, etc; son las actividades específicas que debe proporcionar el sistema.

Los **requerimientos del sistema** varían desde generalidades que cubren lo que tiene que hacer el sistema hasta especificaciones que reflejan maneras locales de trabajar. Tiene que ser completa (deben definirse todos los servicios requeridos por el usuario) y consistente (los requerimientos no deben ser contradictorios entre sí), aunque esto es casi imposible de lograr en sistemas grandes.

Requerimientos no funcionales

Se relacionan con propiedades emergentes del sistema, como fiabilidad, tiempo de respuesta y uso de almacenamiento. También pueden definir restricciones sobre la implementación, como las capacidades de los dispositivos I/O o las representaciones de datos usados en las interfaces con otros sistemas.

Son más significativos que los requerimientos funcionales y el fracaso en cubrir los requerimientos **no funcionales** deriva en la inutilidad del sistema.

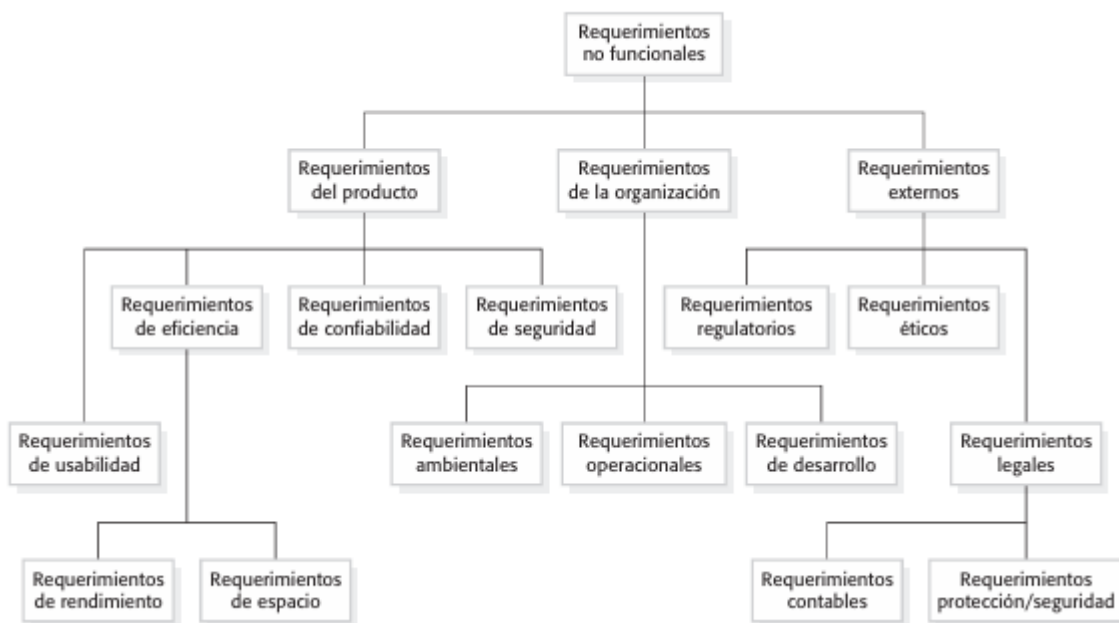
Son difíciles de relacionar con componentes debido a que los requerimientos no funcionales afectan más a la arquitectura global de un sistema que a los componentes individuales y un requerimiento no

funcional *individual* podría generar requerimientos funcionales relacionados que definan nuevos servicios del sistema que se requieran.

Tipos de requerimientos no funcionales

Los requerimientos no funcionales surgen a través de:

1. **Requerimientos del producto:** especifican o restringen el comportamiento del software. Ejemplo: rendimiento, usabilidad, fiabilidad, seguridad, etc.
2. **Requerimientos de la organización:** derivan de políticas y procedimientos de la organización del cliente y del desarrollador. Ejemplo: proceso operacional, de desarrollo, estándares del entorno, etc.
3. **Fuentes externas:** aquellos factores externos al sistema que influyen en él. Ejemplo: requerimientos regulatorios, legislativos, éticos, etc.

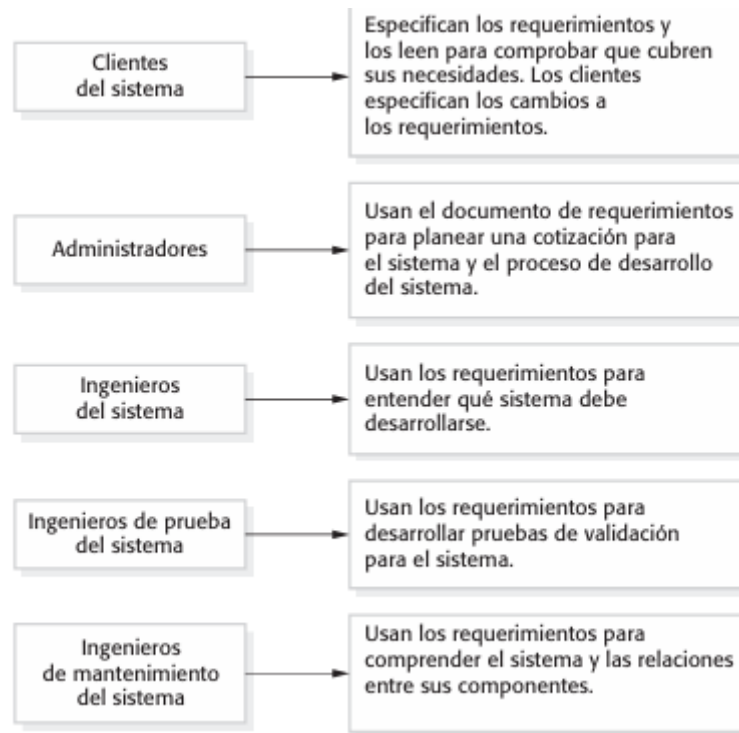


Los requerimientos no funcionales deben escribirse, siempre que sea posible, de manera cuantitativa, de manera que puedan ponerse objetivamente a prueba. Deben utilizarse métricas, como *rapidez*, *tamaño*, *facilidad de uso*, etc. para especificarlos.

El documento de requerimientos de software

Es el comunicado oficial de lo que deben implementar los desarrolladores del sistema. Se recomienda recopilar requerimientos de **usuario** incrementalmente como **historias de usuario** en lugar de un documento formal.

La estructura sugerida del documento incluye: prefacio, introducción, glosario, definición de requerimientos de usuario, arquitectura y especificación de requerimientos del sistema, modelos, evolución del sistema, apéndices e índice.



Especificación de requerimientos

La etapa inicial del ciclo de vida del software es el proceso de escribir el documento de requerimientos, que incluye los **requerimientos de usuario** (descripción funcional y no funcional comprensible para usuarios sin conocimiento técnico) y los **requerimientos del sistema** (versiones extendidas de los anteriores, que detallan *cómo* el sistema cumple los requerimientos). Ambos deben ser claros, sin ambigüedades, completos y consistentes.

Procesos de la ingeniería de requerimientos

Estos procesos incluyen cuatro actividades de alto nivel: estudio de factibilidad, adquisición y análisis, especificación y validación.

En el inicio del proceso, se hará más esfuerzo por comprender los requerimientos empresariales de alto nivel y los no funcionales, así como los requerimientos del usuario para el sistema. A medida que avance, en los anillos exteriores de la espiral, se dedicará más esfuerzo a la adquisición y comprensión de los requerimientos detallados del sistema.

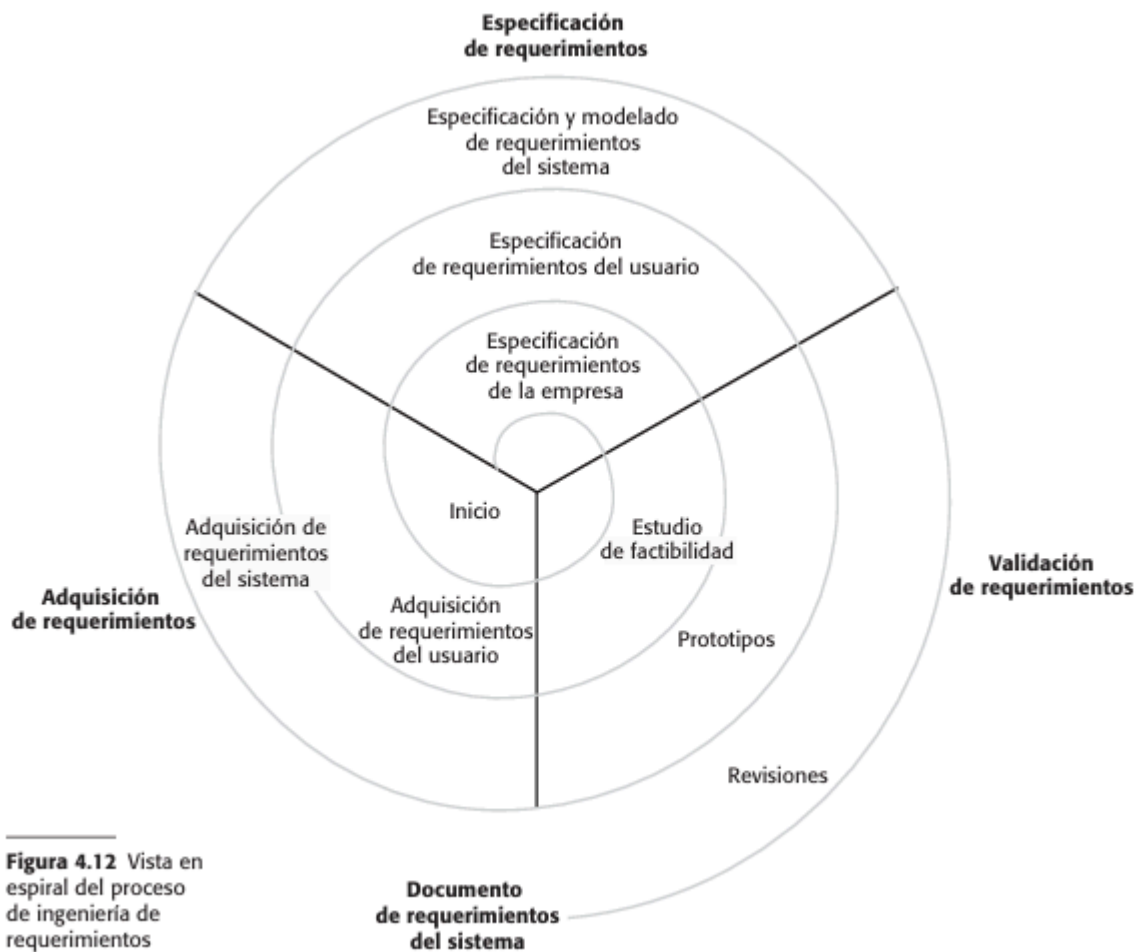


Figura 4.12 Vista en espiral del proceso de ingeniería de requerimientos

Adquisición y análisis de requerimientos

Esta etapa implica interactuar con clientes y usuarios para definir el dominio de la aplicación, los servicios, el rendimiento y las restricciones del sistema.

Las actividades clave son:

1. **Descubrimiento:** Interactuar con los participantes para obtener requerimientos.
2. **Clasificación y Organización:** Agrupar los requerimientos iniciales no estructurados en conjuntos coherentes, a menudo usando un modelo de arquitectura.
3. **Priorización y Negociación:** Priorizar requerimientos en conflicto y resolver desacuerdos mediante negociación.
4. **Especificación:** Documentar los requerimientos para la siguiente fase.



La adquisición de requerimientos es **difícil** porque el cliente puede no saber lo que quiere, la terminología de los participantes puede no ser clara para el equipo de desarrollo, y diferentes usuarios pueden expresar los mismos requerimientos de distintas maneras.

Descubrimiento de requerimientos

Es el proceso de recolección de información sobre los sistemas requeridos y existentes para separar los requerimientos del usuario y del sistema. Las fuentes de información son documentación, participantes, especificaciones de sistemas similares, dominio de la aplicación, y otros sistemas interactuantes. Estas representan distintos puntos de vista del sistema, cada uno mostrando un subconjunto de requerimientos.

Entrevistas

Las entrevistas formales o informales son una parte clave para obtener requerimientos. Estas pueden ser **cerradas** o **abiertas**, aunque generalmente son una combinación de ambas.

La información obtenida en entrevistas debe complementarse con otra información obtenida por un medio diferente.

Escenarios

Se presenta un ejemplo de una interacción (o conjunto pequeño de ellas) y con ellos se busca que los participantes del sistema capten huecos o detalles a incluir en dichos escenarios.

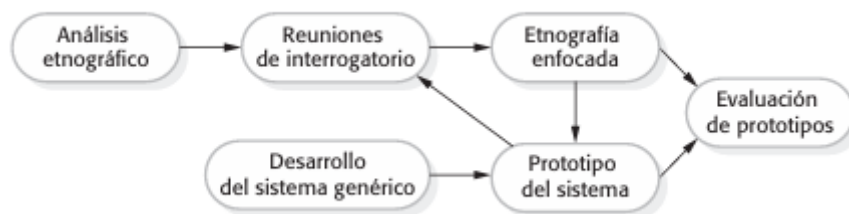
Casos de uso

Este tipo de diagrama identifica interacciones entre el sistema y sus usuarios u otros sistemas mediante **casos de uso**, cada uno documentado textualmente. El conjunto representa todas las interacciones posibles descritas en los requerimientos. Los **actores** son figuras sencillas y las **interacciones** elipses etiquetadas, unidas por líneas. No son útiles para encontrar restricciones, requerimientos empresariales o no funcionales de alto nivel, ni requerimientos de dominio.



Etnografía

La **etnografía** es una técnica de observación para entender procesos operacionales y derivar requerimientos, inmersa en el ambiente laboral. Observa el trabajo diario y las tareas de los participantes para descubrir requerimientos implícitos, factores sociales y organizacionales. Útil para requerimientos derivados de la forma de trabajar y la cooperación, pero menos adecuada para requerimientos organizacionales o de dominio debido a su enfoque en el usuario final.



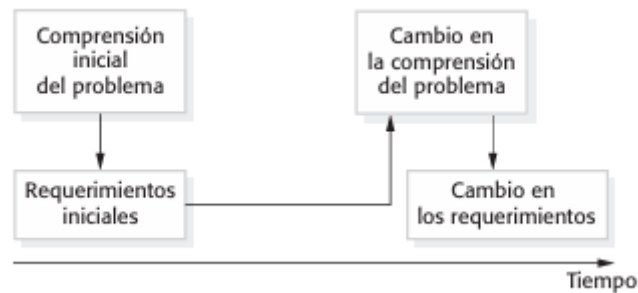
Validación de requerimientos

La validación de requerimientos verifica que se defina el sistema deseado por el cliente, lo cual es crucial, ya que los errores tempranos causan grandes costos si se detectan tarde.

El proceso incluye:

1. **Validez:** Que reflejen necesidades reales del usuario.
2. **Consistencia:** Ausencia de contradicciones.
3. **Totalidad:** Inclusión de todas las funciones y restricciones.
4. **Realismo:** Factibilidad técnica, económica y temporal.

5. **Verificabilidad:** Posibilidad de diseñar una prueba objetiva para cada requerimiento.



Técnicas de validación:

1. **Revisiones:** Análisis de equipo para detectar errores y omisiones.
2. **Prototipos:** Modelo ejecutable para que el usuario valide funciones.
3. **Casos de prueba:** Diseño de pruebas; si es imposible, el requerimiento es inviable o está mal planteado.

Administración de requerimientos

La **administración de requerimientos** es el proceso de comprender y controlar los cambios en los requerimientos del sistema. Es preciso establecer un proceso formal para hacer cambios a las propuestas y vincular estos con los requerimientos del sistema.

Planeación de la administración de requerimientos

Esta etapa establece el nivel de detalle que se requiere. Se debe decidir sobre:

1. **Identificación:** cada requerimiento debe identificarse de manera exclusiva.
2. **Administración del cambio:** se valora el efecto y costo de los cambios.
3. **Políticas de seguimiento:** las relaciones entre requerimientos entre sí y con el diseño del sistema debe registrarse.
4. **Herramientas de apoyo:** Se necesitan para:
 - a. *Almacenamiento:* deben guardarse en un lugar seguro y accesible.
 - b. *Administración del cambio.*
 - c. *Administración del seguimiento.*

Administración del cambio en los requerimientos

Debe aplicarse a todos los cambios en los requerimientos. Es crucial para determinar si los beneficios justifican los costos.

Tiene tres etapas:

1. **Análisis y especificación del problema:** Identificar, analizar el problema y reportarlo para la decisión de implementación.
2. **Análisis del cambio y estimación del costo.**
1. **Implementación:** Modificar el documento de requerimientos y otros artefactos (diseño, código, etc.).

Estudio de factibilidad

La **factibilidad** es una etapa crucial en la ingeniería de software, realizada al inicio, que determina si el desarrollo de un sistema propuesto es viable, efectivo y generará resultados aceptables. Es esencial para la planificación, alineación con objetivos y evitar la inversión en proyectos inviables.

Objetivos del Estudio: Evaluar la **viabilidad** (técnica, económica, operativa, legal), **comprender el problema y requerimientos** para estimar recursos, definir alternativas y establecer una planeación general.

Preparativos Previos:

- **Conformar el Equipo:** Asignar roles, definir líder y establecer un *plan de trabajo*.
- **Organizar la Documentación:** Definir identidad del proyecto y seleccionar técnicas de recolección de información.

Actividades Centrales: 1. Entender el proyecto. 2. Establecer duración y tamaño. 3. Determinar costos y beneficios. 4. Determinar la factibilidad general. 5. **Elaborar un documento con recomendaciones.**

Estructura General: Descripción del proyecto, análisis de mercado, estudio técnico, estudio financiero, evaluación de riesgos y conclusión.

Fases del Proceso: **Recopilación de Datos** (entrevistas), **Duración** (breve), e **Informe de Viabilidad** (definición de problema y síntesis de objetivos).

Dimensiones del Análisis:

1. **Técnica:** ¿Es posible la implementación con los recursos/personal/tecnología disponibles o adquiribles?
2. **Económica:** Evalúa costos vs. beneficios (tangibles e intangibles), usando métodos de comparación (ej. punto de equilibrio).
3. **Operativa:** Predice si el sistema será usado y funcional, centrándose en el diseño y facilidad de uso.
4. **Legal:** Asegura el cumplimiento de leyes y regulaciones.
5. **De Tiempo:** Evalúa si el sistema se puede completar en un plazo útil.

Tablas de decisión

Son una técnica de modelado para especificar la lógica del sistema. Permiten representar sistemáticamente las condiciones de un problema y las acciones asociadas a cada combinación de estas, garantizando una especificación clara y completa.

Estructura

Se divide en cuatro cuadrantes, que se analizan en sentido horario partiendo de la esquina superior izquierda.

- **Condiciones** (superior izquierdo): lista de factores o condiciones que afectan el proceso de decisión.
- **Alternativas de condiciones** (superior derecho): posibles respuestas/valores para cada condición ("S", "N" o un valor específico).
- **Reglas de acción** (inferior derecho): muestra qué combinaciones de alternativas de condiciones disparan acciones específicas. Generalmente se marca con una "X".
- **Acciones** (inferior izquierdo): enumera todos los pasos que deben realizarse como resultado del análisis.

Identificación de Condiciones	Valores de Condiciones
Identificación de Acciones	Valores de Acciones (Reglas de Decisión)

Condición 1	$C_{1,1}$	$C_{1,2}$	...	$C_{1,P}$
...	...	...	...	...
Condición N	$C_{N,1}$	$C_{N,2}$	...	$C_{N,P}$
Acción 1	X			
...				
Acción N		X		

Proceso de desarrollo

1. **Identificar las condiciones y acciones:** determinar todos los factores relevantes y las posibles respuestas.

2. Calcular el tamaño máximo:

- a. **Tablas binarias (S/N):** el número de columnas (reglas) = multiplicar el número de alternativas de cada condición. Ejemplo: 4 cond c/ 2 alt c/u = $2 * 2 * 2 * 2 = 2^4 = 16$ columnas.
 - b. **Tablas extendidas:** cantidad de reglas = multiplicar la cantidad de valores posibles. Ejemplo: Cond 1: 3 valores; Cond 2: 2 valores. Cant de reglas = $3 * 2 = 6$.
 - c. **Tablas mixtas:** cantidad de reglas = 2 (cond. bin) * valores de condiciones no binarias. Ejemplo: Cond 1: 2 valores; Cond 2: 2 valores; Cond 3: 3 valores. Cant de reglas = $2 * 2 * 3 = 12$
3. **Completar y simplificar:** Se insertan las marcas de acción y se combinan las reglas redundantes. Si una alternativa no influye en el resultado, se representa con un guión.

Tipos de tablas de decisión

Según el tipo de entradas (estructura)

1. Limitadas/Binarias

Condiciones: las alternativas son valores binarios (“S”, “N”).

Acciones: Se marcan con una “X” para indicar que el procedimiento debe ejecutarse y se deja en blanco si no.

Uso: Decisiones lógicas sencillas donde los factores son del tipo “se cumple o no se cumple”.

Ejemplo: Encendido de una alarma:

Condiciones / Reglas	R1	R2	R3	R4
Movimiento	S	S	N	N
Sistema activado	S	N	S	N
Acciones				
Activar alarma	X			
No hacer nada		X	X	X

2. Extendidas

Condiciones: valores o rangos específicos.

Acciones: procedimientos específicos o valores calculados.

Uso: las condiciones o acciones tienen más de dos valores posibles.

Ventaja: se reduce el número de filas de condiciones necesarias para representar un problema complejo.

Ejemplo:

Condiciones:

- C1: Monto de compra
 - < \$100.
 - \$100 - \$1000.
 - \$1000
- C2: Tipo de cliente:
 - Regular
 - VIP

Condiciones / Reglas	R1	R2	R3	R4	R5	R6
Monto de Compra	<100	<100	100-1000	100-1000	>1000	>1000
Tipo de cliente	Regular	Vip	Regular	Vip	Regular	Vip
Acciones						
0% descuento	x					
10% descuento		X	X			
20% descuento				X	X	X

Acciones:

- A1: Dcto. 0%.
- A2: Dcto. 10%.
- A3: Dcto. 20%.

3. Mixta

La entrada de condiciones o acciones puede ser limitada o extendida o una combinación de ambos.

Ejemplo:

Condiciones:

- C1: ¿Cliente VIP? → S/N.
- C2: ¿Tiene deudas? → S/N
- C3: Ingreso mensual → Bajo/Medio/Alto

Condiciones / Reglas	R1	R2	R3	R4	R5
Cliente VIP	N	N	---	S	S
Tiene deudas	S	N	N	S	N
Ingreso mensual	--	Bajo	Medio/Alto	Bajo	Medio/Alto
Acciones					
Rechazar	X	X			
Interés alto			X	X	
Interés bajo					x

Acciones:

- A1: Rechazar préstamo.
- A2: Aprobar con interés alto.
- A3: Aprobar con interés bajo.

Según el flujo de decisión (uso)

Abiertas

El resultado envía a otra tabla de decisión para seguir evaluando.

Ejemplo: Sistema de atención médica.

Tabla 1: Evaluación inicial

Condición /Regla	R1	R2
Condición	Si	No
¿Tiene fiebre?	A	B

A → Ir a Tabla 2 (síntomas respiratorios).

B → Ir a Tabla 3 (otros síntomas).

Tabla 2: Síntomas respiratorios.

Condiciones:

- C1: ¿Tiene tos? S/N
- C2: ¿Tiene dificultad para respirar? S/N

Acciones:

- A1: Posible gripe.
- A2: Posible infección respiratoria grave.
- A3: Observación.

Condiciones / Reglas	R1	R2	R3	R4
Tos	S	S	N	N
Dificultad respiratoria	S	N	S	N
Acciones				
Infección grave	X			
gripe		X		
Observación			X	X

Tabla 3: Otros síntomas

Condiciones:

- C1: ¿Tiene dolor abdominal? S/N.
- C2: ¿Tiene vómitos? S/N.

Acciones:

- A1: Problema digestivo leve.
- A2: Posible intoxicación.
- A3: Sin urgencia.

Condiciones / Reglas	R1	R2	R3	R4
Dolor abdominal	S	S	N	N
Vómitos	S	N	S	N
Acciones				
Intoxicación	X			
Problema leve		X		
Sin urgencia			X	X

Cerradas

El resultado es una acción **final**. Ejemplos vistos en la clasificación por entrada.

Consideraciones

- **Integridad y precisión:** se deben incluir todos los valores posibles en las entradas extendidas para evitar que la tabla esté incompleta (si hay un rango de 100 a 1000, debe haber uno de 0 a 100, si corresponde).
- **Simplificación:** fusionar reglas redundantes. Se utiliza un guion para señalar que una alternativa es indistinta o no afecta el resultado específico de esa regla.
- **Detección de errores:** verificar la lógica porque al mezclar tipos de entrada aumenta el riesgo de tener reglas idénticas con acciones distintas.

Simplificación de tablas

- **Eliminación de redundancias:** Hay redundancia cuando *dos reglas de decisión son idénticas* (salvo para una condición del renglón) *y las acciones para ambas son idénticas*.
- **Supresión de contradicciones:** Hay contradicciones cuando dos o más reglas tienen el mismo conjunto de condiciones, *pero sus acciones son diferentes*.

Ejemplo:

Condiciones/Reglas	R1	R2	R3	R4	R5	R6	R7	R8
Pago	S	S	S	S	N	N	N	N
Stock	S	S	N	N	S	S	N	N
VIP	S	N	S	N	S	N	S	N
Acciones								
Enviar	X	X						
No enviar			X	X			X	X
Prioridad Alta	X							
Revisar					X	X		

Los casos (R3 R4), (R5 R6) y (R7 R8) tienen los mismos resultados. Entonces, se suman casos, agrupando los que coincidan en todo menos en un parámetro.

Tabla simplificada:

Condiciones/Reglas	R1	R2	R3 – R4	R5-R6	R7-R8
Pago	S	S	S	N	N
Stock	S	S	N	S	N
VIP	S	N	-	-	-
Acciones					
Enviar	X	X			
No enviar			X		X
Prioridad Alta	X				
Revisar				X	

Árbol de decisión

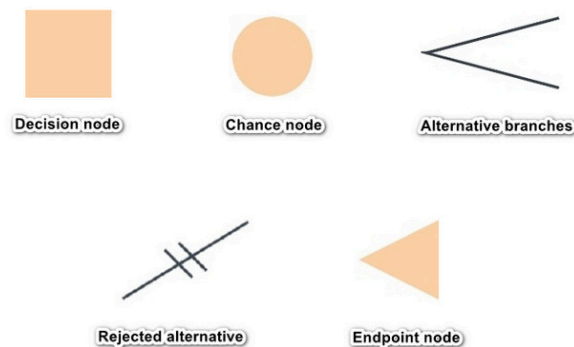
Método de análisis para decisiones estructuradas que debe mantener una cadena de decisiones en una secuencia específica.

Estructura y notación

Se dibujan de costado, con la raíz del lado izquierdo y las ramas se extienden por el derecho.

Su **simbología**:

- **Nodos de probabilidad:** Círculos que describen una condición (si).
- **Nodos de decisión:** Cuadrados que describen una acción procedimental (entonces).
- **Ramas:** Conectan los nodos y describen alternativas posibles para cada decisión.
- **Nodos terminales:** Triángulo que describe el resultado de la secuencia lógica.



Aplicaciones principales

- **Análisis de la lógica de procesos:** Identifican y organizan las condiciones y acciones de procesos estructurados.
- **Administración de proyectos:** Evaluar rutas alternativas: construir el sw de cero, reutilizar componentes, subcontratar, etc. El árbol incluye probabilidades y costos proyectados en cada opción.
- **Minería de datos:** Hallar patrones complejos en grandes volúmenes de datos.

Criterio de selección

- Permite observar de inmediato el orden en que se verifican las condiciones y se ejecutan las acciones.
- Es ideal cuando no todas las condiciones son relevantes para todas las acciones, permitiendo rutas separadas.
- Efectivos para comunicarse con personas ajenas al equipo técnico.

Ventajas

- Fácil de entender.
- Requiere poca cantidad de preparación de datos y no hace falta estandarizarlos.
- Se pueden procesar datos cualitativos y cuantitativos.
- Puede procesar grandes cantidades de datos.
- Se puede combinar fácilmente con otra herramienta de toma de decisiones.

Desventajas

- Pueden contener muchos datos inútiles, por lo que hay que definir la cantidad máxima de nodos.
- Pueden ser inestables, ajustar un pequeño conjunto de datos puede generar grandes cambios.

Proceso de desarrollo

1. Identificar todas las condiciones y acciones, junto a su orden crítico y sincronización.
2. Comenzar la construcción de izquierda a derecha y enlistar todas las posibles alternativas de una rama antes de pasar a la siguiente.
3. Enumerar los nodos para facilitar su referencia.

Ejemplo:

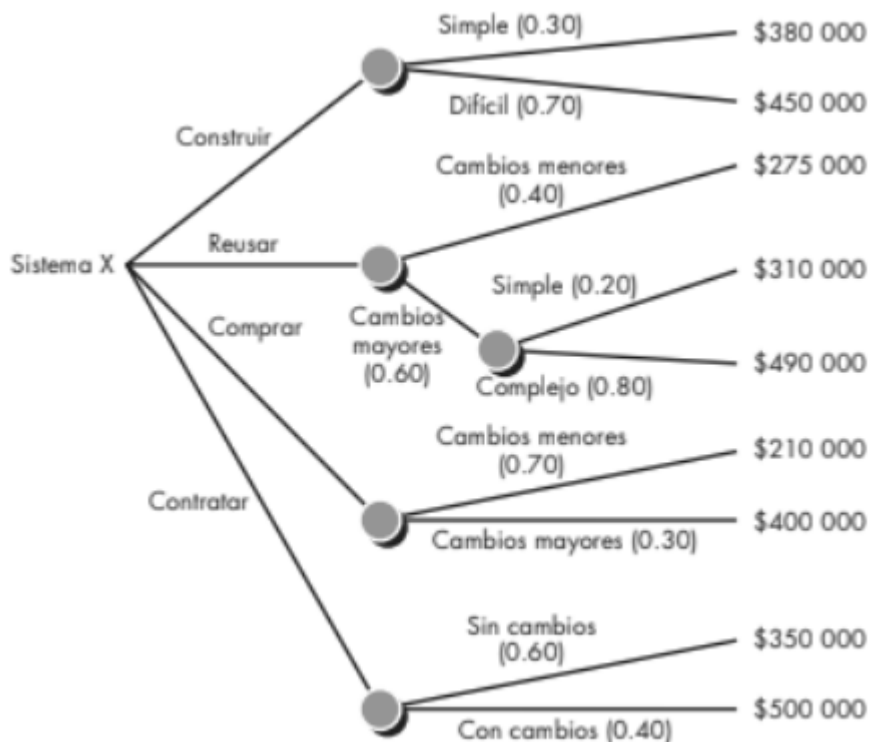


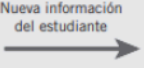
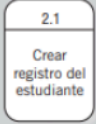
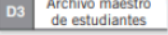
Diagrama de flujo de datos

Es una técnica de **análisis estructurado** para representar la entrada, los procesos, la salida y el almacenamiento de datos.

Su **objetivo** es descomponer un problema complejo en componentes más sencillos y manejables, facilitando el mantenimiento y la comprensión de la lógica de negocio.

Se desarrollan con la metodología arriba-abajo, avanzando de lo general a lo específico.

	Yourdon, DeMarco	Gane y Sarson	SSADM METRICA
Flujo de Datos			
Procesos			
Almacén de Datos			
Entidades Externas			

Símbolo	Significado	Ejemplo
	Entidad	
	Flujo de datos	
	Proceso	
	Almacén de datos	

Elementos

1. **Entidad externa:** origen o destino de datos. Interactúa con el sistema pero es externa a este. Se le asigna un nombre y puede usarse más de una vez en el DFD para evitar que las líneas se crucen. Ejemplos: banco, empresas, etc.
2. **Proceso:** Se simboliza con un círculo (Yourdan) y es el componente que representa cualquier función que transforma flujos de datos de entrada en uno o varios flujos de datos de salida.

Se representan con un número y un nombre (ambos únicos) representativo según su función con la forma: verbo + sustantivo. Pueden o no expandirse, dependiendo de su complejidad. Si no se expande, es un **proceso primitivo**.
3. **Flujo de datos:** Representa datos en movimiento. La flecha indica el movimiento de los datos de un punto a otro. Si varios flujos de datos ocurren simultáneamente, se pueden escribir con flechas paralelas.

La flecha lleva nombre, ya que representa los datos de una persona, lugar o cosa.
4. **Almacén:** Representa datos en reposo, la información temporal del sistema. Puede aparecer varias veces en el DFD para mejorar la legibilidad. Se representan en el nivel 2.

Estructura

1. NIVEL 0: Diagramas de contexto

Es el nivel más alto y general. Representa el sistema como un solo proceso (el 0) y muestra sólo sus interacciones con entidades externas.

Define los límites y el alcance del sistema (cuáles procesos/información son internos y cuáles externos).

2. NIVEL 1: Subsistemas

Descompone el diagrama de contexto, dividiéndolo en subprocesos internos, mostrando sus cambios y la circulación de la información dentro del sistema.

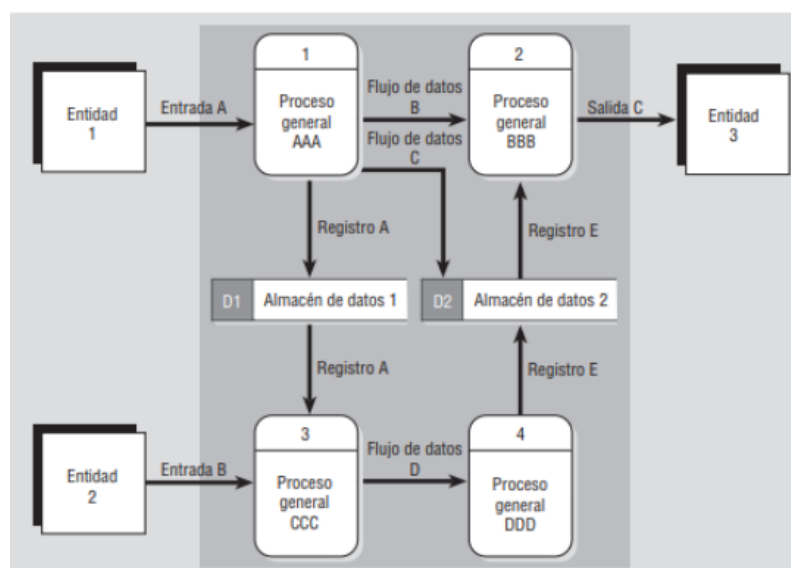
3. NIVEL 2: Funciones de cada subsistema

Descompone cada subsistema, con la intención de especificar las funciones internas y subprocesos que componen cada uno o un proceso mayor identificado en el nivel anterior.

Conexiones permitidas entre componentes de un DFD

	PROCESO	ALMACÉN	ENTIDAD EXTERNA
PROCESO	SI	SI	SI
ALMACÉN	SI	NO	NO
ENTIDAD EXTERNA	SI	NO	NO

Ejemplo



Objetivos, límites y alcances del DFD

Los **objetivos** de un diagrama se determinan al responder a la pregunta: *¿Cuál es el fin que se busca lograr?*

Los **límites** demarcan el sistema de su entorno, especificando qué procesos y datos integran la aplicación y cuáles corresponden al exterior. Se visualizan en el **diagrama de contexto (nivel 0)**, indicando el inicio (desde) y el fin (hasta) del flujo de datos.

El **alcance** se restringe a las *actividades de negocio* y su función es distinguir los requerimientos reales, focalizando el diseño en las funcionalidades que el sistema debe gestionar internamente.

En un DFD, el alcance se establece delimitando qué procesos, datos y actores forman parte del sistema y cuáles quedan fuera. Los componentes que definen el alcance incluyen:

- **Límites del sistema:** Lo que del mundo real se representa (*qué*), usualmente en el nivel 0.
- **Entradas y salidas:** Los datos que ingresan y la información que el sistema genera.
- **Entidades externas:** Actores o sistemas que interactúan con el sistema en cuestión, pero que no lo integran.
- **Procesos incluidos:** Solamente aquellos que se encuentran *dentro* de la definición del sistema.

La definición clara del alcance es crucial porque previene malentendidos, optimiza la comunicación y establece con precisión el ámbito de trabajo.

Diccionario de datos

Es una obra de consulta que contiene metadatos. Recopila y coordina términos específicos, confirmando su significado y asegurando la limpieza y consistencia de los datos.

Objetivos y utilidad

Es una *herramienta de análisis* que sirve para:

- Validar la integridad y precisión de los DFD.
- Proveer un punto de partida para el diseño de pantallas e informes.
- Determinar el contenido de los datos almacenados en archivos y DB.
- Desarrollar la lógica para los procesos del DFD.
- Gestionar nombres de manera consistente.

Categorías de entradas

1. **Flujo de datos:** Representan datos en movimiento. Influyen nombre descriptivo, origen, destino, volumen por unidad de tiempo y la estructura de datos que viaja con el flujo.
2. **Estructuras de datos:** Grupos de elementos relacionados. La notación puede ser algebraica o BNF.
3. **Elementos de datos:** Es la unidad más pequeña de información (atributo o campo). Tienen alias, longitud, tipo de dato, valores predeterminados y criterios de validación.

4. **Almacenes de datos:** Representan datos estáticos. Tienen ID, nombre, alias, tipo de archivo y estructura de datos que contienen.

Notación utilizada

- **Algebraica:**
 - = está compuesto de.
 - + y.
 - {} elementos repetitivos.
 - [] selección de alternativas.
 - () elementos opcionales.
- **BNF:**
 - [|] selección obligatoria.
 - {}* iteración n veces.
 - @ señala campos clave o identificadores.

Ejemplo: Sistema de venta y reserva de pasajes

1. ELEMENTOS DE DATOS (datos atómicos)

- | | |
|--|---|
| • DNI: Numérico (8) – Identificación del pasajero | • CodigoReserva: Alfanumérico |
| • Nombre: Alfanumérico (30) | • EstadoReserva: {Activa Cancelada Vencida} |
| • Apellido: Alfanumérico (30) | • PrecioBase: Decimal |
| • Fecha: Fecha (DD/MM/AAAA) | • MontoFinal: Decimal |
| • Hora: Hora (HH:MM) | • PorcentajeDescuento: {0 15 20} |
| • Origen: Alfanumérico | • TipoDescuento: {IdaVuelta Jubilado Estudiante} |
| • Destino: Alfanumérico | • PorcentajeReintegro: {80 50 25} |
| • TipoServicio: {Cama SemiCama Ejecutivo} | • MontoReintegro: Decimal |
| • NumeroBoleto: Numérico (ID único) | |

2. ESTRUCTURAS DE DATOS (notación algebraica)

PASAJERO

PASAJERO = @DNI + Nombre + Apellido

VIAJE

VIAJE = Origen + Destino + Fecha + Hora + TipoServicio

PASAJE

PASAJE = @NumeroBoleto + VIAJE + PrecioBase + MontoFinal

RESERVA

RESERVA = @CodigoReserva + PASAJERO + VIAJE + EstadoReserva

DESCUENTO

DESCUENTO = TipoDescuento + PorcentajeDescuento

REINTEGRO

REINTEGRO = NumeroBoleto + PorcentajeReintegro + MontoReintegro

VENTA

VENTA = PASAJERO + PASAJE + [DESCUENTO]

REPORTE_DIARIO

REPORTE_DIARIO = Fecha +
TotalRecaudado + CantidadPasajes +
TotalReintegros

3. FLUJOS DE DATOS

SOLICITUD_COMPRA
SOLICITUD_COMPRA = PASAJERO +
VIAJE + [CodigoReserva]

- Origen: Pasajero
- Destino: Sistema

CONFIRMACION_COMPRA
CONFIRMACION_COMPRA =
NumeroBoleto + Estado + MontoFinal

EMISION_BOLETO
EMISION_BOLETO = PASAJE +
[DESCUENTO]

SOLICITUD_RESERVA
SOLICITUD_RESERVA = PASAJERO +
VIAJE

CONFIRMACION_RESERVA
CONFIRMACION_RESERVA =
CodigoReserva + EstadoReserva

CANCELACION
CANCELACION = NumeroBoleto

REINTEGRO_PASAJE
REINTEGRO_PASAJE = REINTEGRO

SOLICITUD_REPORTE
SOLICITUD_REPORTE = Fecha

REPORTE_RECAUDACION
REPORTE_RECAUDACION =
REPORTE_DIARIO

ALMACENES DE DATOS

ARCHIVO_PASAJES
ARCHIVO_PASAJES = {PASAJE}

ARCHIVO_RESERVAS
ARCHIVO_RESERVAS = {RESERVA}

ARCHIVO_VENTAS
ARCHIVO_VENTAS = {VENTA}

ARCHIVO_REINTEGROS
ARCHIVO_REINTEGROS = {REINTEGRO}